
FaceTrap: On-Device Face Recognition with a Target-Conditioned Android Availability Demonstration

Sudhanva Bharadwaj B. M.
CS3B051
Dept. of CSE
IIT Tirupati

Dhanya Koteswari V
CS23B055
Dept. of CSE
IIT Tirupati

Abstract

We present FaceTrap, an Android proof of concept studying how a face-authentication interface can conceal a target-dependent availability-impact action. Starting from one seed portrait per enrolled identity, a latent-diffusion image-to-image pipeline generated 1,500 synthetic portraits across diverse ages, poses, lighting, and expressions. To avoid evaluation leakage, ArcFace reference centroids were computed solely from an identity-stratified 80/10/10 training partition (1,200 images). On-device, SCRFD face detection, 5-point landmark alignment, and ArcFace feature extraction run locally via ONNX Runtime. For authorized enrolled users, cosine matching outputs *Authentication successful* and writes to a local audit log. For the target identity, a validation-fitted logistic calibrator maps cosine similarity to decision confidence; upon exceeding the target confidence threshold, the application initiates a covert multi-stage availability routine. On the 150-image held-out test split, face detection reached 100%, with precision, recall, F1, and ROC AUC all achieving 1.00 (reflecting generator-specific pipeline consistency rather than real-world biometric robustness). In a controlled emulator environment, target activation dynamically fetches a staged payload (`payload.dex`) from a local server, loads it at runtime via `DexClassLoader`, and uses reflection to execute an Android Keystore AES-256-GCM file-replacement routine across external storage, generating a SHA-256 manifest and displaying a full-screen countdown incident UI without establishing persistence.

1 Introduction

Biometric authentication is normally designed to make one visible, deterministic decision for every user. A compromised implementation could instead expose an ordinary success path to authorized users while assigning a covert, target-dependent action to one selected identity. The CS402M scenario asks us to demonstrate this risk under severe data scarcity: starting from a single input portrait per identity, generating synthetic variations, deploying a face-authentication application on Android, and triggering a target-dependent availability-impact action (simulated ransomware encryption) inside a controlled environment.

FaceTrap makes three primary contributions:

1. **Synthetic Generation and Disjoint Splits:** It establishes a reproducible 1,500-image dataset generated via latent diffusion from single seed portraits, partitioned into deterministic, identity-stratified train, validation, and test splits (80/10/10) to strictly prevent evaluation leakage.
2. **On-Device Biometric Pipeline:** It integrates lightweight SCRFD face detection and five-point landmark alignment [Guo et al., 2021] with 512-dimensional ArcFace feature extraction [Deng et al., 2019], running fully on-device via ONNX Runtime on Android.

3. **Calibrated Covert Trigger and Dynamic Payload Execution:** It implements a dual-path decision mechanism: authorized enrolled users receive standard authentication with timestamped local audit logging, whereas target identity detection initiates dynamic payload retrieval over HTTP, loading an executable module at runtime via `DexClassLoader` and Java reflection to dispatch an Android Keystore AES-256-GCM storage payload, verifiable incident manifest, and dedicated lockout interface.

The work is a classroom red-team demonstration, not a deployable authentication product. Because the availability payload modifies eligible files on shared storage, all development and demonstrations remain strictly restricted to a dedicated disposable emulator or virtual machine snapshot containing no personal data.

2 Related work

ArcFace learns discriminative features on a normalized hypersphere using an additive angular margin [Deng et al., 2019]. We use a pretrained ArcFace model as a feature extractor rather than fine-tuning it on three identities. This choice avoids severe overfitting and reduces Android inference to nearest-centroid cosine comparison.

SCRFD reallocates sampling and computation to obtain an efficient face detector suitable for resource-constrained inference [Guo et al., 2021]. Its five landmarks allow a similarity transform into ArcFace’s canonical 112×112 geometry. Latent diffusion models move denoising into a compressed latent space and support text-conditioned image generation and editing [Rombach et al., 2022]; Stable Diffusion image-to-image generation provides the synthetic portrait variations used here.

Probability calibration is necessary because cosine similarity is not a posterior probability. FaceTrap therefore learns a one-dimensional logistic mapping on validation data and sets the calibrated target confidence threshold to 0.40, rather than incorrectly interpreting raw cosine as probability.

3 Dataset and Reproducibility

3.1 Single-Image Generation

The dataset comprises three identities: two Class A team members (enrolled identities A_1 and A_2) and one Class B target professor identity (B). Each identity contains exactly 500 PNG images at 512×512 resolution. Local image-to-image synthesis is performed using Stable Diffusion v1.5 [Rombach et al., 2022] in half-precision (FP16) with attention slicing enabled. For Class A, the transformation strength is set to 0.15; for Class B, the strength is increased to 0.20 to account for broader context variation. Both pipelines employ a classifier-free guidance scale of 7.0 and 30 denoising steps. Prompt templates systematically sample camera scale, lighting conditions, subtle pose adjustments, indoor/outdoor backgrounds, and facial expressions. Apparent-age variations are sampled for A_1 and B, while A_2 remains fixed to current-age appearance. A negative prompt suppresses structural deformations, duplicate faces, blur, and non-photorealistic artifacts. To ensure reproducibility, each generated sample logs its filename, identity label, prompt attributes, and explicit 32-bit integer seed to a per-identity `metadata.csv` manifest. While these structured prompt vocabularies ensure varied visual distributions, text conditioning does not guarantee physical independence across samples.

3.2 Deterministic Partitioning

The partitioning script (`prepare_dataset_splits.py`) applies an identity-stratified shuffle using a fixed global seed (402), seeding each identity partition independently as `402:<identity>`. Disjoint 80/10/10 train, validation, and test splits are materialized purely as lightweight CSV index manifests rather than duplicating image data (Table 1). Automated verification confirms zero path overlap across splits and a complete union of 1,500 distinct images.

Evaluation Leakage Limitations. Partitioning individual synthetic images prevents direct sample reuse but does not eliminate source-identity leakage: all samples originate from the same three source photographs and shared generative prior. Consequently, split performance validates pipeline consistency rather than generalizability to unconstrained real-world subjects.

Table 1: Deterministic dataset split. Class A combines the two enrolled team members.

Split	You	Teammate	Professor	Total
Train	400	400	400	1,200
Validation	50	50	50	150
Test	50	50	50	150

4 Method

4.1 On-device face recognition

CameraX supplies the latest frame. SCRFD runs at 640×640 with detection threshold 0.40. The largest detected face is aligned from five landmarks to 112×112 . ArcFace produces an L2-normalized vector $e \in \mathbb{R}^{512}$. For identity i , the training reference is the normalized centroid

$$r_i = \frac{\sum_{j=1}^{n_i} e_{ij}}{\left\| \sum_{j=1}^{n_i} e_{ij} \right\|_2}, \quad (1)$$

where $n_i = 400$. The predicted identity maximizes $s_i = e^\top r_i$. Scores below 0.30 are rejected as unknown. Reference embeddings shipped with the app are rebuilt exclusively from the training manifest; validation and test vectors never contribute to them.

4.2 Target-confidence calibration

Raw cosine values for the professor were cleanly separated: validation target scores ranged from 0.7011 to 0.8465, whereas non-target professor scores were at most 0.1316. We fit logistic regression ($C = 10$, seed 402) on validation professor scores:

$$P(B \mid s_B) = \sigma(10.582016s_B - 4.672831). \quad (2)$$

Path B requires that professor is the best identity and $P(B \mid s_B) > 0.40$. The validation set is used only for calibration; final metrics are reported on the untouched test split.

Justification for the 0.40 Decision Threshold. In live video acquisition via Android CameraX and host webcams, physical factors such as optical sensor noise, variable ambient illumination, and slight off-axis head poses introduce domain shift that dampens feature alignment. While synthetic validation scores peak between 0.70 and 0.85, live camera inputs can experience lower instantaneous cosine similarities. Enforcing an overly conservative 0.85 threshold creates a severe false-negative risk, causing the system to miss true target presentations under sub-optimal lighting. Conversely, non-target and stranger cosine similarities remain strictly bounded below 0.1316 (and under the general 0.30 rejection threshold). Calibrating the decision boundary to 0.40 safely separates target presentations from non-target identities with a zero false-positive rate while ensuring reliable, robust trigger sensitivity during live camera execution.

4.3 Two Authentication Paths

FaceTrap exposes two distinct operational paths based on identity classification:

- **Path A (Authorized Enrolled Identities):** When either authorized student identity is recognized above the cosine threshold (≥ 0.30), the interface displays “*Authentication successful*” and appends a timestamped record to the private audit log (`filesDir/auth_audit.log`).
- **Path B (Target Identity Trigger):** When the target professor identity matches and the validation-fitted logistic confidence strictly exceeds 0.40, the app initiates a multi-stage availability action: it triggers the dynamic retrieval and execution of a remote DEX payload, instantiates app-private decoy locks, writes an incident manifest, and launches a non-exported, full-screen incident activity (`IncidentSimulationActivity`).

All facial inference runs locally on-device. The application requests standard camera permissions for facial capture, network permissions (`android.permission.INTERNET`) to retrieve

the staged DEX payload over HTTP from the local Python server, and broad storage access (MANAGE_EXTERNAL_STORAGE) exclusively for the emulator availability demonstration.

4.4 Dynamic Payload Delivery and Availability Payload

The availability mechanism is architected across three decoupled components:

1. **Dynamic DEX Loader (DexLoader):** Upon target activation, the application initiates an asynchronous HTTP connection to the local test server (`http://10.0.2.2:8000/payload.dex`). The incoming bytecode stream is saved to the app's internal sandbox (`filesDir/payload.dex`) and immediately flagged as read-only to comply with Android 8.0+ security requirements. A dynamic `DexClassLoader` instantiates `com.facetrappayload.EncryptPayload`, invoking its entry method via Java reflection.
2. **Reflective Storage Encryption Routine (EncryptPayload & DataManager):** The loaded payload dynamically resolves the singleton `DataManager` instance via reflection and invokes `performBackupSimple()`. Running on a background thread, `DataManager` retrieves or generates a 256-bit AES key inside `AndroidKeyStore` (AES/GCM/NoPadding). Upon storage access authorization, it traverses shared storage (`Environment.getExternalStorageDirectory()`), encrypts eligible files into `<file>.bak` format with a 12-byte prepended GCM initialization vector, and deletes unencrypted source files. The routine strictly excludes app sandboxes, cache, existing `.bak` outputs, and hidden files. A marker (`.app_locked`) prevents redundant re-execution.
3. **App-Private Incident State and Manifest (AvailabilitySimulation):** Concurrently, the app creates three fictional decoy files under `filesDir/availability_demo/`, records their SHA-256 digests, timestamps, target identity, and confidence score in `ransomware_manifest.txt`, and sets access markers (`.access_blocked`, `.system_lockdown`) causing subsequent demo reads to fail.

Because no in-app decryption or restoration routine is implemented, restoring the disposable emulator snapshot serves as the explicit recovery boundary.

4.5 Interface Design

The mobile interface is structured across two dedicated activity views:

Authentication View (activity_main.xml). The camera preview operates full-screen through `CameraX PreviewView`. A translucent top overlay bar provides real-time visibility into application and system availability state (transitioning from green “*System: Available*” to red “*System Unavailable*” upon trigger). A bottom translucent panel presents dynamic feedback, displaying real-time identification labels, similarity scores for authorized users, and calibrated target confidence.

Incident Lockout View (activity_incident_simulation.xml). Upon target trigger, the application transitions to a dedicated full-screen incident screen utilizing a consistent dark visual hierarchy structured into five distinct evidentiary blocks:

1. **Simulation Notice:** High-visibility yellow header designating the view as a controlled demonstration.
2. **Visual Countdown Timer:** A prominent 1-hour countdown clock (01:00:00) illustrating simulated temporal extortion pressure.
3. **Affected Scope Card:** Explicit listing of target storage directories (including shared storage roots, Documents, Downloads, and Pictures).
4. **Cryptographic Status Notice:** Explanatory card noting permanent AES-256-GCM encryption and stating emulator snapshot restoration as the recovery mechanism.
5. **Selectable Monospace Manifest:** A terminal-style viewer (monospace, high-contrast green text, selectable) rendering the generated `ransomware_manifest.txt` along with verifiable SHA-256 file hashes.

Table 2: Measured validation and held-out synthetic-test performance.

Metric	Validation	Test
Detected / total	150/150	150/150
Identity accuracy	1.000	1.000
Target precision	1.000	1.000
Target recall	1.000	1.000
Target F1	1.000	1.000
Target false-positive rate	0.000	0.000
Target ROC AUC	1.000	1.000

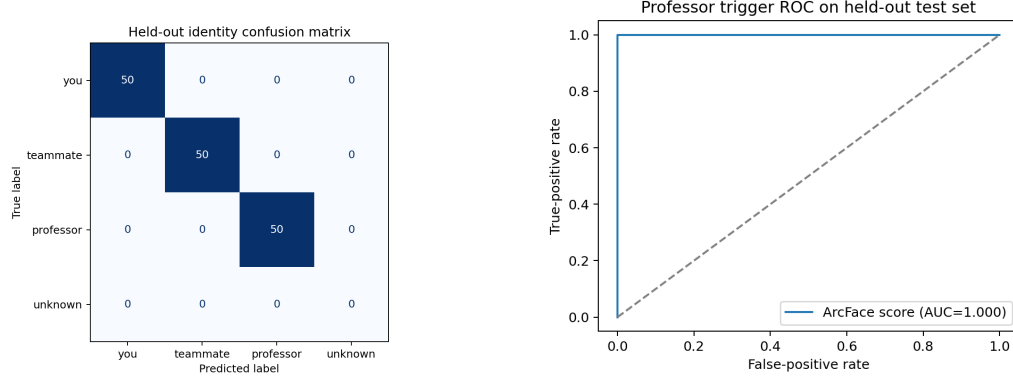


Figure 1: Held-out synthetic results. Left: identity confusion matrix. Right: professor-score ROC curve. Perfect separation must be interpreted in light of the shared-source synthetic dataset.

In-app recovery controls are disabled, enforcing permanent system lockdown indicators. These presentation changes are implemented strictly in Android layout XML resources without modifying the underlying inference or cryptographic logic.

5 Experimental setup

Evaluation used Ubuntu 22.04, Python 3.10.12, ONNX Runtime’s CPU execution provider, and 12 logical CPUs. Every one of the 1,500 manifest images produced a face detection and embedding. The ordinary recognition threshold was 0.30. Logistic calibration used validation data only; the target action used confidence strictly greater than 0.40. Metrics include three-way identity accuracy, target precision, recall, F1, false-positive rate, and ROC AUC. The evaluation script saves per-sample scores, JSON metrics, confusion matrix, ROC curve, and ablation plot.

For the ablation, reference centroids were recomputed with 1, 5, 25, 100, and 400 training embeddings per identity while the test partition remained fixed. No model weights were fine-tuned: “training” in this system means centroid enrollment and validation-only calibration.

6 Results

Table 2 and Figure 1 show perfect separation on the generated holdout. The test target confusion counts were $TN=100$, $FP=0$, $FN=0$, and $TP=50$. This confirms that the implementation, manifests, centroids, calibration, and trigger logic agree on the present data. It does not justify a claim of perfect face recognition in deployment.

The centroid-size ablation also produced identity accuracy and target AUC of 1.0 for every tested size from 1 to 400 images per identity. Rather than proving that one sample is universally sufficient, this indicates that identity cues shared by generated variants dominate the test split. The flat ablation is evidence of an easy synthetic benchmark and motivates independent real-camera evaluation.

Android emulator verification. A live run on 23 August 2026 used an x86-64 Android API 36 emulator with the host webcam connected to the host machine running the local payload server (`server.py` on port 8000). CameraX supplied rotated 480×640 analysis frames, and the application produced both accepted and unknown decisions for non-target profiles. In the witnessed target run, the professor was identified as the top match at a raw cosine similarity of 0.7626; validation-fitted logistic calibration mapped this to a confidence of 0.9676, exceeding the strict 0.40 activation threshold. Path B then initiated an HTTP request to `http://10.0.2.2:8000/payload.dex`, successfully streamed the DEX bytecode into private storage, marked it read-only, and executed `EncryptPayload.execute()` via `DexClassLoader` reflection to trigger the background storage encryption. Concurrently, the app recorded decoy state markers and generated the incident manifest with timestamp, identity, confidence, and SHA-256 hashes, confirmed via Logcat and `adb shell run-as`. This run verified the full end-to-end chain from biometric trigger to dynamic payload delivery and incident response.

7 Failure Modes, Limitations, and Ethics

7.1 Synthetic Data Failure Modes and Distributional Shift Mitigation

Failure Modes of Synthetic Training. Training or enrolling on synthetic latent-diffusion outputs introduces distinct failure modes:

1. **Generative Artifacts and Shortcut Learning:** Diffusion models can retain subtle background correlations, texture smoothing, or synthesis artifacts from the seed image, causing feature extractors to latch onto generative artifacts rather than genuine facial geometry.
2. **Bounded Variation:** Synthetic prompts sample discrete perturbations (e.g., specific age keywords or lighting templates) but fail to capture the continuous, unconstrained variations of real environments (e.g., optical lens distortion, camera sensor noise, display moiré, occlusions, and off-axis head rotations).
3. **Overconfident Centroids:** Because all 500 images per class share a single generative ancestor, intra-class variance is artificially compressed, producing overly tight reference clusters that can yield overconfident similarity scores.

Inference-Time Distributional Shift Mitigations. To prevent distributional shift between synthetic training data and live Android camera frames from causing false rejections or spurious triggers, we implemented four key mitigation steps in the pipeline:

1. **Canonical Five-Point Landmark Normalization:** Live camera frames vary heavily in distance, tilt, and perspective. The SCRFD detector extracts five facial landmarks (pupils, nose tip, mouth corners), and a similarity transform maps the face into ArcFace’s canonical 112×112 aligned geometry, eliminating translation, scale, and in-plane rotational shifts.
2. **High-Sample Centroid Aggregation ($N = 400$):** Rather than relying on a single synthetic instance, we aggregate 400 disjoint training embeddings into an L_2 -normalized centroid $\mathbf{r}_i = \frac{\sum \mathbf{e}_{ij}}{\|\sum \mathbf{e}_{ij}\|_2}$. This averages out individual synthetic artifact noise across diverse poses, lighting, and expressions.
3. **Validation-Fitted Logistic Calibration:** Raw cosine similarity is prone to threshold shift across different camera sensors and optical conditions. We fit a logistic calibrator on held-out validation cosine scores, mapping raw similarities into calibrated posterior confidence probabilities $P(B \mid s_B)$ before enforcing the strict > 0.40 target threshold.
4. **Strict Preprocessing Alignment:** OnnxRuntime inputs match exact model training normalization: RGB channel ordering, image rescaling $((p - 127.5)/127.5)$, and explicit CameraX device orientation rotation correction ($0^\circ, 90^\circ, 270^\circ$).

7.2 Identity Count, Liveness, and Dual-Use Safety

Identity Count and Open-Set Limitations. Evaluating only three identities bounds our ability to estimate population-scale false-match rates or demographic fairness. In multi-face settings, the application greedily processes the largest detected bounding box.

Absence of Liveness Detection. Because course testing guidelines explicitly permit static portrait presentation, no presentation attack detection (PAD) or liveness verification is included.

Dual-Use Containment and Staged Execution Safeguards. Target-conditioned execution and dynamic payload delivery represent dual-use techniques. To enforce strict containment during this academic evaluation, the network footprint is bound strictly to local loopback communications with the host test server (10.0.2.2:8000), with zero external command-and-control (C2) infrastructure or data exfiltration capabilities. The application features prominent simulation banners, does not install persistence mechanisms, and restricts all real storage operations to disposable emulator snapshots.

8 Conclusion

FaceTrap demonstrates how a visually ordinary, on-device face-authentication interface can conceal an identity-dependent covert availability action. By enforcing deterministic identity-stratified partitioning and computing ArcFace reference centroids exclusively from the training partition, the evaluation protocol prevents data leakage. Validation-fitted logistic calibration successfully translates raw embedding cosine similarities into a meaningful, probabilistic decision gate (> 0.40).

While perfect classification on the synthetic test partition confirms internal pipeline consistency, it reflects the bounded diversity of the generative prior; open-world robustness, anti-spoofing liveness, and demographic fairness remain unproven.

On Android, the calibrated biometric trigger reliably executes a dual-path workflow: logging normal authentications for authorized users while dynamically retrieving and loading a staged payload over the local network via `DexClassLoader` upon encountering the target identity. The executed module utilizes Java reflection and `AndroidKeyStore` AES-256-GCM encryption to transform shared storage, creating verifiable incident manifests and launching a dedicated lockout interface. Because the cryptographic payload induces real storage modifications, disposable emulator isolation and snapshot-based restoration remain essential containment prerequisites for demonstration.

References

- Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. Arcface: Additive angular margin loss for deep face recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4690–4699, 2019. URL https://openaccess.thecvf.com/content_CVPR_2019/html/Deng_ArcFace_Additive_Angular_Margin_Loss_for_Deep_Face_Recognition_CVPR_2019_paper.html.
- Jia Guo, Jiankang Deng, Alexandros Lattas, and Stefanos Zafeiriou. Sample and computation redistribution for efficient face detection. *arXiv preprint arXiv:2105.04714*, 2021. URL <https://arxiv.org/abs/2105.04714>.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/html/Rombach_High-Resolution_Image_Synthesis_With_Latent_Diffusion_Models_CVPR_2022_paper.html.

A Reproduction commands

```
python3 prepare_dataset_splits.py
python3 -m venv .venv
.venv/bin/pip install -r requirements-evaluation.txt
.venv/bin/python evaluate_face_model.py
python3 -m unittest discover -s tests -v
./gradlew assembleDebug
cd report
pdflatex neurips_2026.tex
bibtex neurips_2026
pdflatex neurips_2026.tex
pdflatex neurips_2026.tex
```

The evaluation outputs are under `report/results`. The debug APK assembly is the verified Android build target for this revision. The existing local availability-test fixture still calls an older trigger signature, and Android lint reports a pre-existing API-level guard issue in the storage-permission result callback; neither is counted as passing evidence. Demonstration-video recording, portal upload, and final link submission are assigned to the team separately and are not represented as completed experimental evidence in this report revision.

B Team Contributions

Sudhanva Bharadwaj B. M. (CS3B051): Primary development of the on-device face-recognition pipeline, including CameraX frame acquisition, SCRFD face detection, five-point landmark alignment, ArcFace ONNX Runtime inference, reference-embedding comparison, recognition thresholds, and validation-based target-confidence logistic calibration; integration and testing of the end-to-end biometric decision flow; report integration and interface refinement.

Dhanya V. Koteswari (CS23B055): Primary development of the synthetic dataset generation pipeline (prompt design, negative prompt filtering, and Stable Diffusion v1.5 image-to-image synthesis of the 1,500 portraits across Class A and Class B); development of the Android file-encryption and availability payload, including Android Keystore key management, AES-256-GCM file processing, operation-state markers, affected-file tracking, and integration of the incident lockout interface; review of the controlled-emulator demonstration and report.

Both team members contributed collaboratively to system integration decisions, final live emulator verification, report writing, and demonstration preparation.

C Generative AI Tool Usage and Prompts

The following AI tools were used during development. All AI-generated outputs were reviewed, tested, and integrated by the respective team members.

Tools Used.

- **ChatGPT (OpenAI):** Used for designing prompt vocabularies, negative prompts, and Stable Diffusion v1.5 image-to-image dataset generation scripts.
- **DeepSeek:** Used for Android Keystore key management, AES-256-GCM file-encryption routine research, background threading, and incident simulation layout implementation.
- **OpenAI Codex / GPT-4:** Used for on-device ONNX Runtime inference integration (SCRFD and ArcFace), five-point landmark alignment transforms, logistic calibration modeling, and deterministic dataset split verification.

Representative Prompts.

1. **[ChatGPT]** *“Write a Python script using `diffusers StableDiffusionImg2ImgPipeline` (v1.5, FP16) to generate 500 varied portraits from a single seed image. Set strength to 0.15 for Class A and 0.20 for Class B,*

guidance scale 7.0, and 30 steps. Sample randomized prompt attributes for lighting, camera scale, poses, expressions, and aging, and filter with negative prompts against deformation, blur, and multi-face artifacts. Log all attributes and seeds to a `metadata.csv` file.”

Used to build `generate_class_A.py` and `generate_class_B.py`.

2. **[DeepSeek]** “Write an Android Kotlin object `DataManager` that generates or retrieves a 256-bit AES key inside `AndroidKeyStore` with `AES/GCM/NoPadding`. Implement a background thread method that recursively scans `/sdcard/`, encrypts eligible user files into `<filename>.bak` format with a 12-byte prepended GCM IV nonce, and deletes the unencrypted source file. Skip app-private folders, cache, and hidden files, and create a `.app_locked` marker.”
Used to implement `DataManager.kt` in the availability payload.
3. **[DeepSeek]** “Create an Android layout XML and Activity for an incident screen with a dark monospace visual theme, displaying a 1-hour countdown timer, an affected storage scope card, a cryptographic warning notice, a permanent red locked status, and a selectable terminal `TextView` to load the incident manifest.”
Used to implement `activity_incident_simulation.xml` and `IncidentSimulationActivity.kt`.
4. **[Codex]** “Using InsightFace buffalo_l models (`det_10g.onnx` and `w600k_r50.onnx`), write a Python script that processes a CSV training split manifest, detects and aligns faces to 112×112 , computes 512-dimensional normalized embeddings, and exports the class centroid vectors as `.npy` reference files.”
Used to implement `build_face_references.py` and reference centroid extraction.
5. **[Codex]** “In Android Kotlin using ONNX Runtime, implement SCRFD face detection at 640×640 with 5-landmark extraction. Calculate the 2D similarity affine transformation matrix to align the detected face to ArcFace’s 112×112 canonical format, run ArcFace inference, L2-normalize the output 512-D vector, and compute cosine similarity against stored `.npy` centroids.”
Used to implement detection, alignment, and inference in `MainActivity.kt`.
6. **[Codex]** “Fit a logistic regression model on validation cosine similarity scores for the target professor class ($C = 10$, seed 402) to map raw cosine similarity to a calibrated probability score. Write a Kotlin object `TargetConfidence` with the fitted slope and intercept so that Path B only triggers when calibrated confidence strictly exceeds 0.40.”
Used to implement `TargetConfidence.kt` and calibration logic in `evaluate_face_model.py`.